

Datsci 224

**Understanding Artificial Intelligence:
From Mathematics to Applications**

Lecture 1:

Probability, Data, and Estimation

Jean Feng

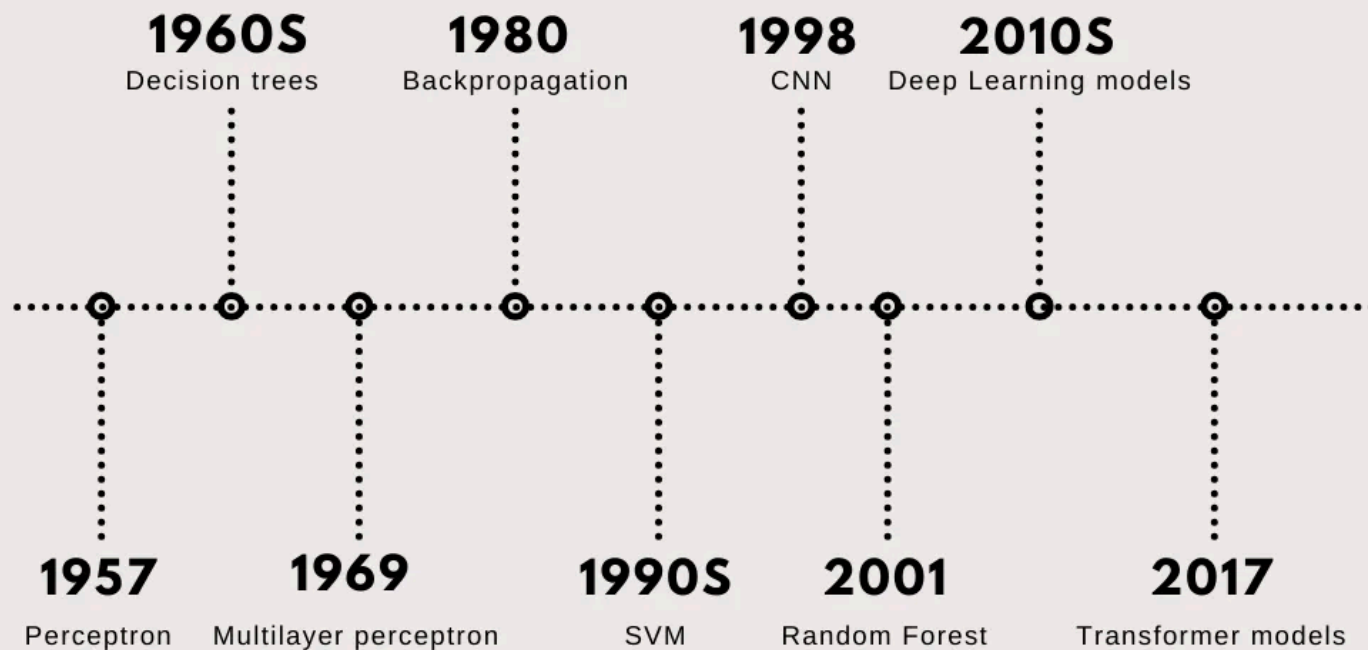
Department of Epidemiology and Biostatistics

Goal of the course:
Teach the mathematical
foundations of AI/ML

Why?

AI/ML moves fast

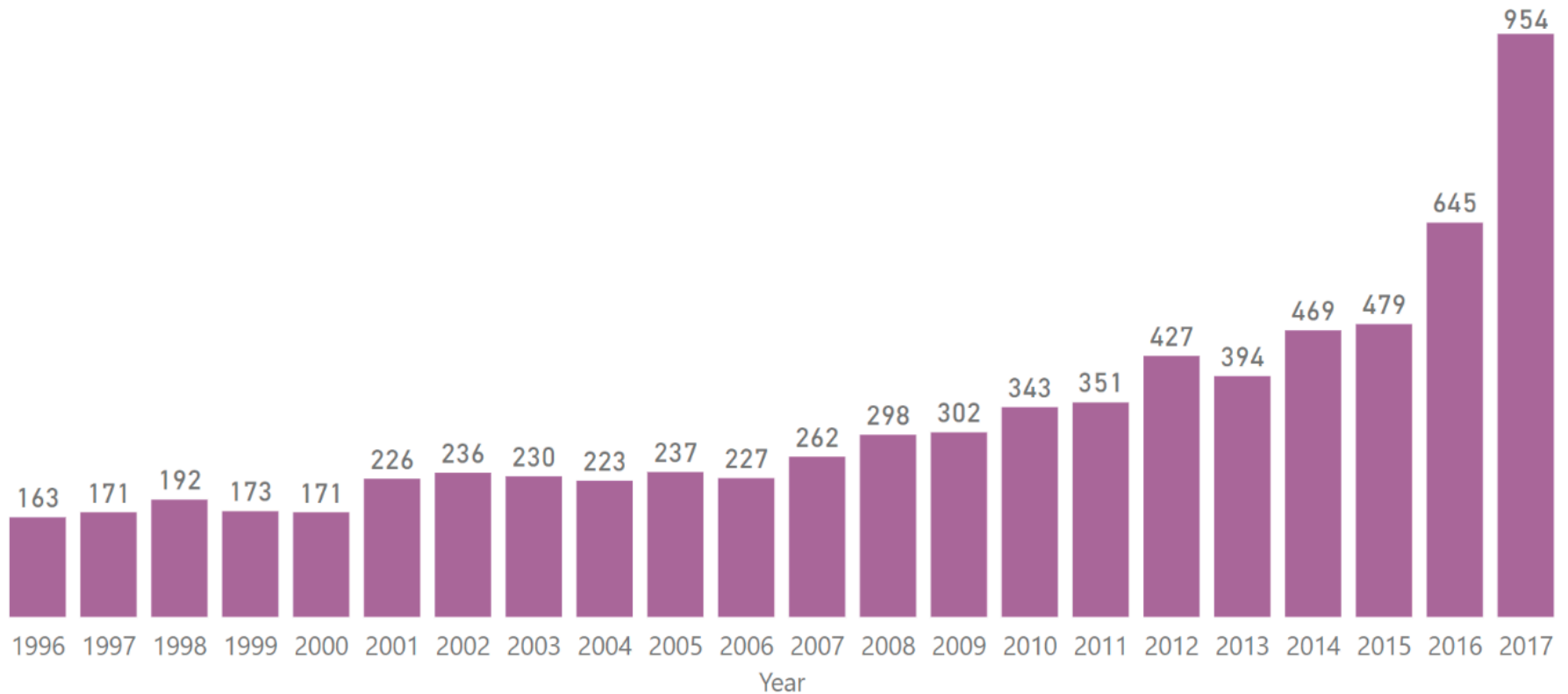
Important ML milestones



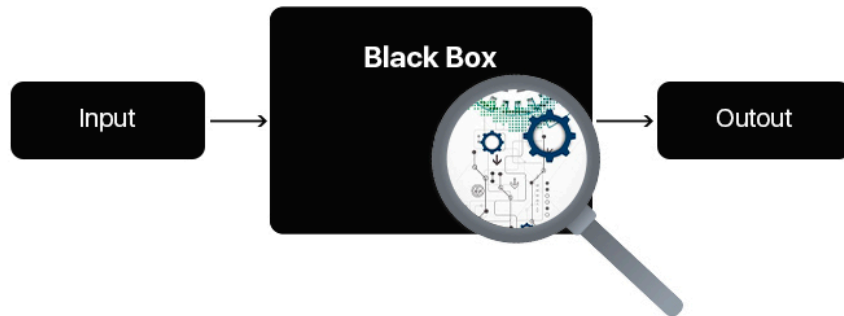
@manzonikie

AI/ML moves fast

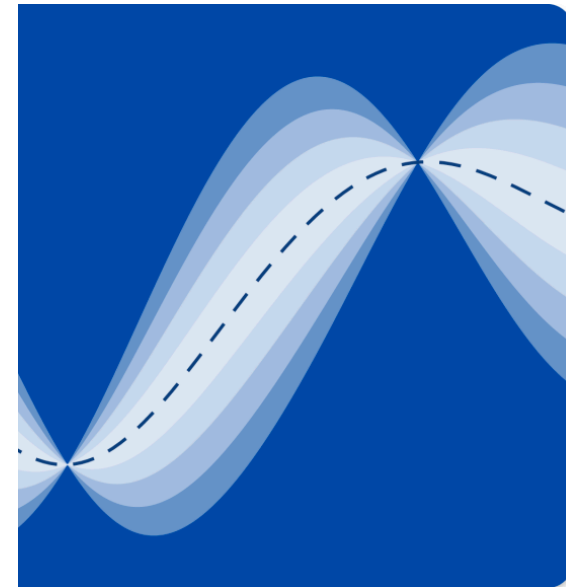
Number of publications at NeurIPS



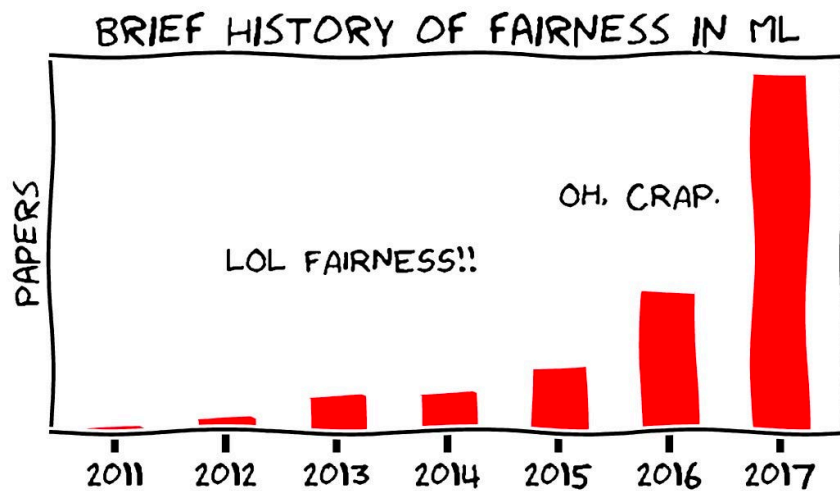
AI/ML moves fast



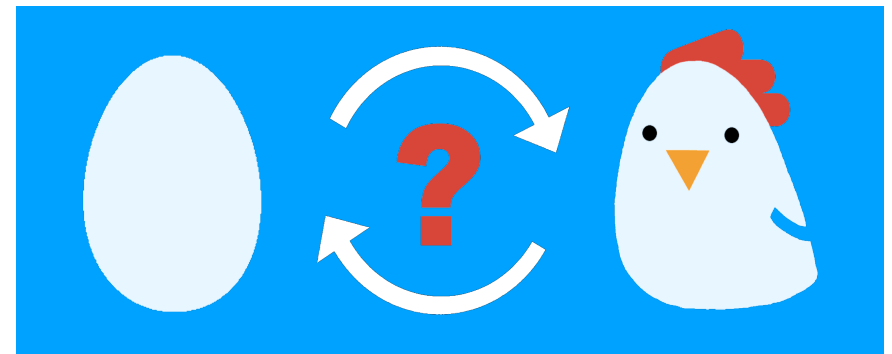
Explainable AI



Uncertainty quantification



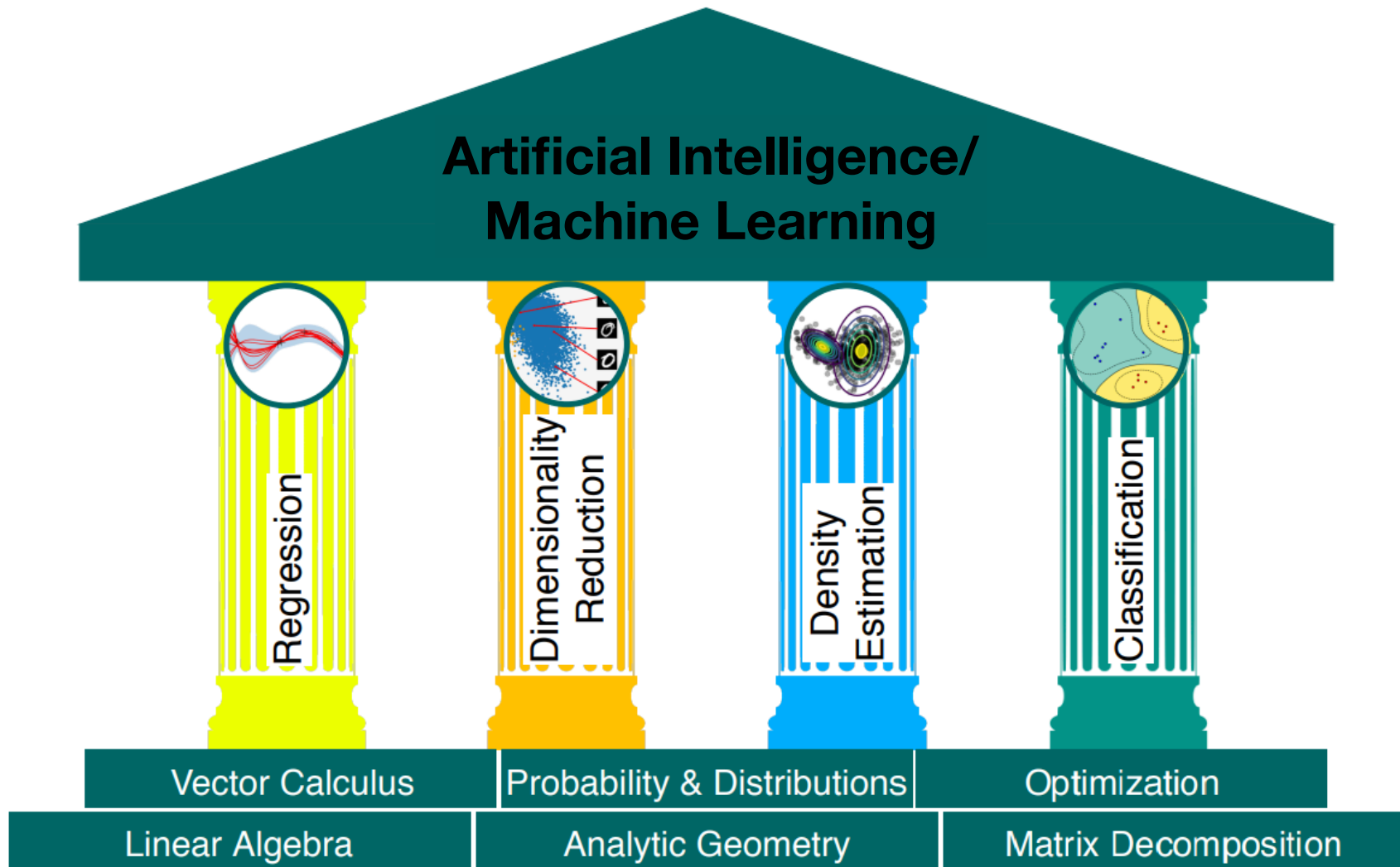
ML Fairness



ML for Causal inference

**I won't be able to teach you
all of AI/ML in one quarter...**

But there are underlying mathematical principles!



Goal of the course

- By the end, the goal is to be able to:
 - **Demonstrate proficiency in learning and applying new AI/ML algorithms.**
 - Explain the key mathematical ideas that underly different algorithms.
 - Select the most appropriate machine learning algorithm/analysis strategy to answer your question of interest.
 - Critique and analyze applications of machine learning algorithms.



Learning
How to Learn

[https://www.coursera.org/
learn/learning-how-to-
learn](https://www.coursera.org/learn/learning-how-to-learn)

Format of the course

- **Lectures:** Introduce a new AI/ML algorithm and dive into the mathematical details.
- **Labs:** We will discuss a AI/ML algorithm that is different from what we learned in lecture but builds on what we learned. Some math, some coding.
- **CLE:** All course materials will be available through the CLE
 - Detailed course syllabus is online
 - Use the course forum to post questions, share reading materials, etc

Format of the course

Lecture Title	Lecture Content
Week 1: Probability, Data, and Estimation	Maximum Likelihood Estimation Probability distributions, Linear Algebra
Week 2: Linear regression	Linear regression, Uncertainty quantification Convex optimization, Gradient descent
Week 3: Regularization	High-dimensional data, Penalized linear regression Constrained optimization, Lagrange Multipliers, Bias-variance tradeoff
Week 4: Ensemble algorithms	Ensembling, Bayesian model averaging, Gradient Boosting, Variable importance Bayesian inference, Optimization
Week 5: Principal Components Analysis	Principal components analysis, Probabilistic PCA Linear algebra, EM algorithm, Variational Inference
Week 6: Deep learning	Neural networks, Neural network modules Linear algebra, optimization, derivatives, backpropagation
Week 7: Deep(er) learning	Convolutional neural networks, recurrent neural networks, transformers
Week 8: Deep(est) learning	Foundation models

Assignments

- **Homeworks:** There will be four homework assignments (Jupyter Notebook files), which have a mix of math and coding.
- **Final project:** Apply a new machine learning algorithm to analyze a dataset.
 - **Check-in:** Submit which ML algorithm you will be using. This will help us coordinate presentations. If multiple students are using the same ML algorithm, you will be asked to coordinate your presentations to present on different aspects.
 - **Slide submission:** Submit the slides you will be presenting.
 - **Presentation:** Present a brief tutorial of the algorithm at the end of the course.
 - **Final report:** A 3-page written report describing the research problem, method, and results.
- *No late assignments will be accepted.*
- All assignments will be submitted through the CLE.

Assignments

Assignment	Due date	Weight
Homework 1	4/13	10%
Homework 2	4/27	10%
Homework 3	5/11	10%
Homework 4	5/25	10%
Project: check-in	5/25	2.5%
Project: presentation slides	One day before assigned presentation date	2.5%
Project: presentations	6/1 or 6/8	25%
Project: final report	6/15	25%
Course Participation	6/8	5%

You will be assigned to a presentation date based on your chosen algorithms.
Make sure to keep the presentation dates open!

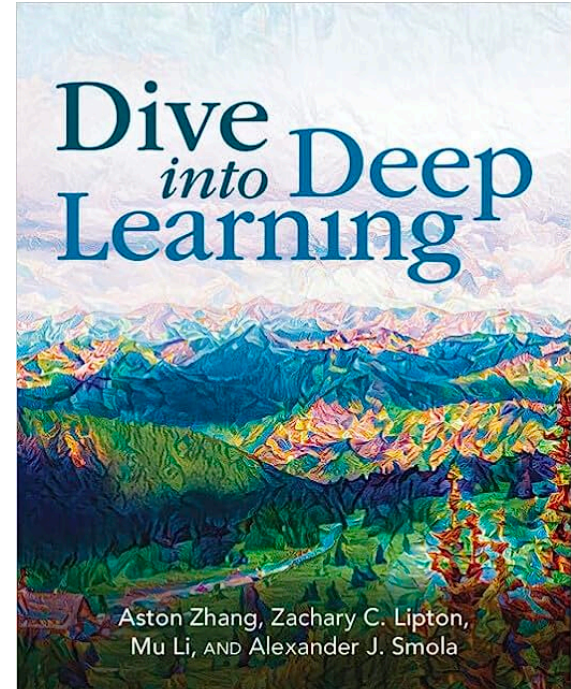
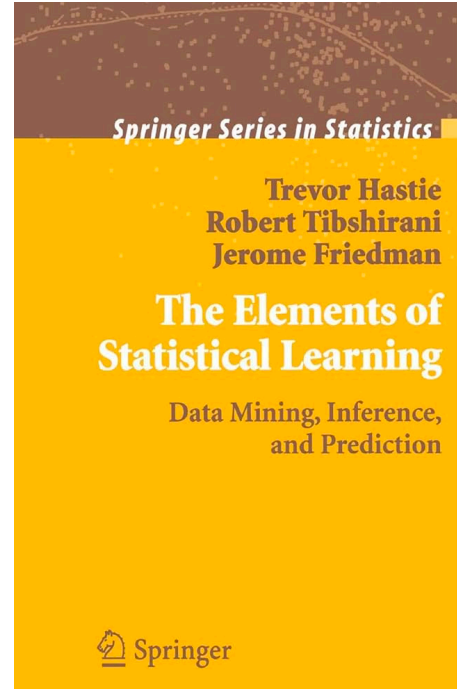
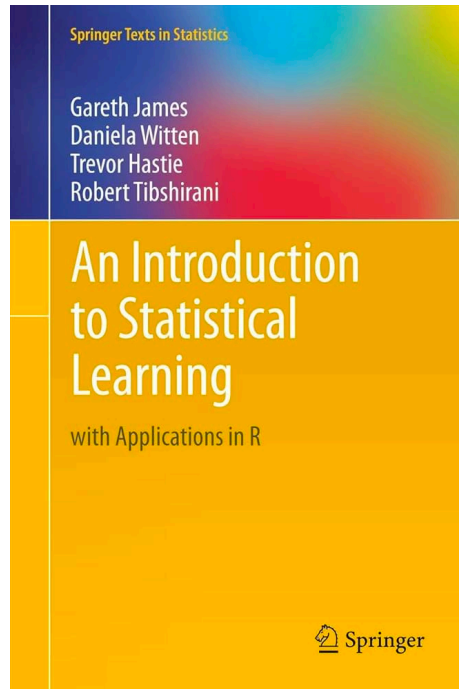
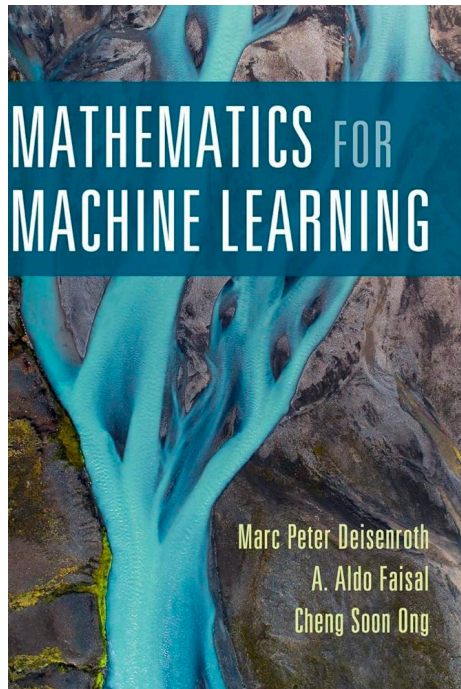
Python

- All homeworks and labs require Python.
- Recommended python version: Python 3.8+
- Recommended python package manager: pip
- Recommended Integrated Development Environment (IDE):
 - Visual Studio Code: <https://code.visualstudio.com/>
 - We'll also use it to run Jupyter Notebooks

Math

- Use Latex to write math for labs, homeworks, and the final project.
- To write $e = mc^2$, we type `$e = mc^2$`.
- To write $\sqrt{\frac{1}{2}}$, we type `$$\sqrt{\frac{1}{2}}$`.
- https://www.overleaf.com/learn/latex/Mathematical_expressions
- Note that you can type math expressions in Latex in Jupyter Notebooks.

Course textbooks



All of these textbooks are freely available online

Learning materials

- This course draws on a lot of materials from diverse sources. We will not be learning from a single textbook or only textbooks.
- There are **many** resources online for AI/ML. You are encouraged to browse other materials to find one that helps you gain a deeper understanding.
 - Wikipedia, Coursera, Youtube videos, Blog posts, Chat-GPT, etc.
 - If you find something really helpful, please post to the forum!
- But keep in mind: Think critically! Not all online materials are correct (and some even hallucinate). There are widespread misconceptions about certain concepts, so think critically about what you read and double-check their claims by cross-referencing other materials.

TICR Professional Conduct Statement

- I will maintain the highest standards of academic honesty
- I will not use answer keys from prior years
- Problem Sets and Projects: I am permitted to consult with other classmates and generative AI technologies, but I will write final answers *on my own in my own words*.

Final words before we dive in

- Please ask questions! Class is meant to be interactive :)
- Feedback on the course is always welcome!
- This class likely has a lot more math than other classes you've taken at UCSF. Get excited!

Lecture Outline

- **Mathematical foundations of machine learning**
- Back to basics:
 - What is data? How is it generated?
 - How do we describe multiple observations?
- What is probabilistic modeling?
- Maximum likelihood estimation
 - Closed-form solutions

Artificial Intelligence/ Machine learning

- AI/ML is the study of computer/statistical algorithms that learn relationships between variables in the data and generalize to unseen data.
- Examples of AI/ML algorithms:
 - Logistic regression
 - Random forests
 - Deep learning
 - Chat-GPT

It's all just math.

Example: AI/ML-based clinical decision support tools

Article

Health system-scale language models are all-purpose prediction engines

<https://doi.org/10.1038/s41586-023-06160-y>

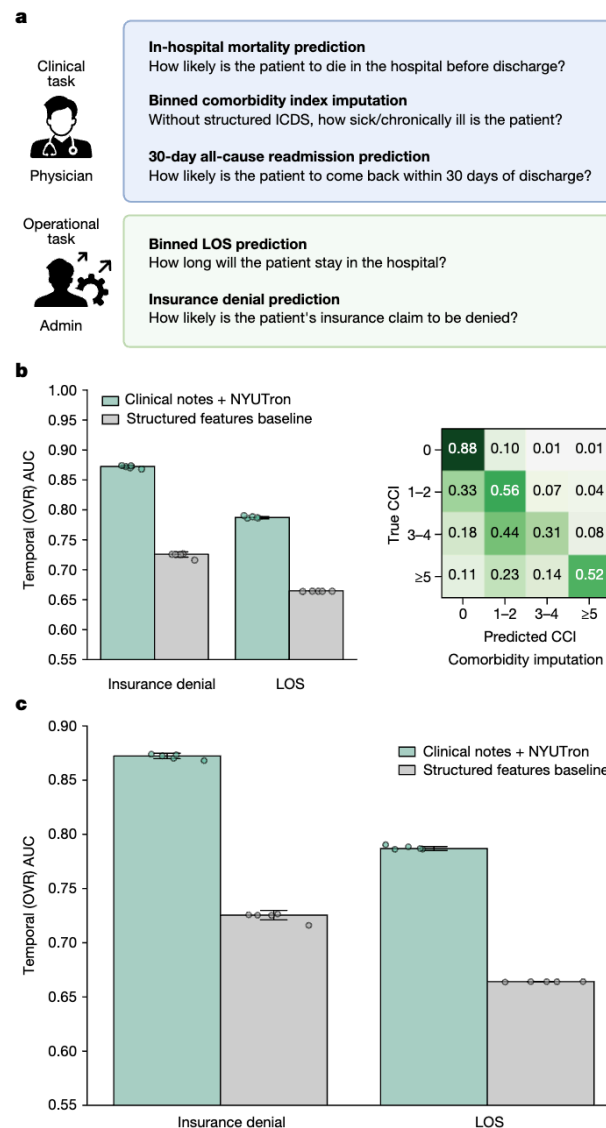
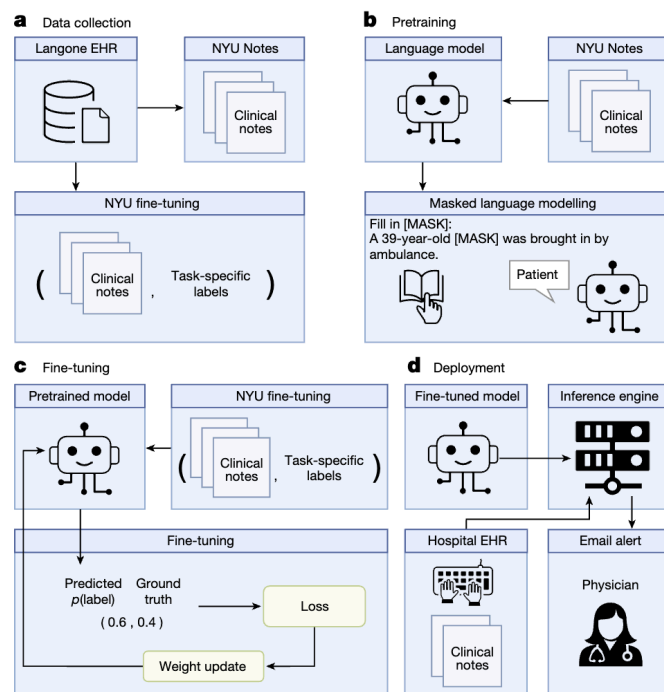
Received: 14 October 2022

Accepted: 2 May 2023

Published online: 07 June 2023

Open access

Lavender Yao Jiang^{1,2}, Xujin Chris Liu^{1,3}, Nima Pour Nejatian⁴, Mustafa Nasir-Moin¹, Duo Wang⁵, Anas Abidin⁴, Kevin Eaton⁶, Howard Antony Riina¹, Ilya Laufer¹, Paawan Punjabi⁶, Madeline Miceli⁶, Nora C. Kim¹, Cordelia Orillac¹, Zane Schnurman¹, Christopher Livia¹, Hannah Weiss¹, David Kurland¹, Sean Neifert¹, Yosef Dastagirzada¹, Douglas Kondziolka¹, Alexander T. M. Cheung¹, Grace Yang^{1,2}, Ming Cao^{1,2}, Mona Flores⁴, Anthony B. Costa⁴, Yindalon Aphinyanaphongs^{5,7}, Kyunghyun Cho^{2,8,9,10} & Eric Karl Oermann^{1,2,11}✉



<https://www.nature.com/articles/s41586-023-06160-y>

The math behind this algorithm

Pretraining

We pretrained a 109 million-parameter BERT model using preprocessed NYU Notes and the MLM objective for 3 weeks (96 epochs) on 24 NVIDIA A100 GPUs distributed over three compute nodes until the validation loss started to plateau. The model has 12 hidden layers with dimension 768, with 12 attention heads per layer. We used a per-device training batch size of 64 and saved every 2,000 steps. We used the Zero Redundancy AdamW optimizer (an improvement over the Adam optimizer) with a constant learning rate of 5×10^{-5} , FP16 mixed precision and stage 2 parallelization^{34–36}.

Fine-tuning

NYUTron+ discharge notes for readmission prediction. We replaced the trained MLM classifier with a randomly initialized linear classifier after the last hidden layer of the pretrained BERT model. We fine-tuned the model end to end using the training set of the NYU Readmission dataset for ten epochs, evaluating the validation AUC every half epoch and stopping early with a patience of five. We used the following hyperparameters from manual tuning based on the validation AUC: a learning rate of 2×10^{-5} , a weight decay of 0.01 and a per-device batch size of 4. We optimized the cross-entropy loss using the AdamW optimizer. While varying the size of the dataset ($N \in \{10^2, 10^3, 10^4, 10^5, 3.92336 \times 10^5\}$), we fine-tuned the pretrained model using subsamples of the NYU Readmission dataset and evaluated their AUC on the temporal test set. For each size of subsample, we ran five experiments with distinct random seeds (0, 13, 24, 36, 42). For comparison, we looked at the median AUC and the standard deviation of the five experiments.

Example: AI/ML-based medical devices

Pivotal trial of an autonomous AI-based diagnostic system for detection of diabetic retinopathy in primary care offices

Michael D. Abràmoff^{1,2,3,4}, Philip T. Lavin⁵, Michele Birch⁶, Nilay Shah⁷ and James C. Folk^{1,2,3}

The AI system exceeded all pre-specified superiority endpoints... Based on these results, FDA authorized the system for use by health care providers to detect more than mild DR and diabetic macular edema, making it, the first FDA authorized autonomous AI diagnostic system in any field of medicine, with the potential to help prevent vision loss in thousands of people with diabetes annually.



The math behind this algorithm

Automatic Detection of Red Lesions in Digital Color Fundus Photographs

Meindert Niemeijer*, Bram van Ginneken, *Member, IEEE*, Joes Staal, *Member, IEEE*, Maria S. A. Suttorp-Schulten, and Michael D. Abràmoff, *Member, IEEE*

Pixel classification is a supervised method, so it needs to be trained using example pixels. Each of the example pixels was extracted from a training set for which a labeled reference standard is available. All pixels in the training set are, thus, assigned a label and a feature vector. A classifier can now establish a decision boundary in the feature space which will optimally separate the two classes (i.e., foreground and background pixels).

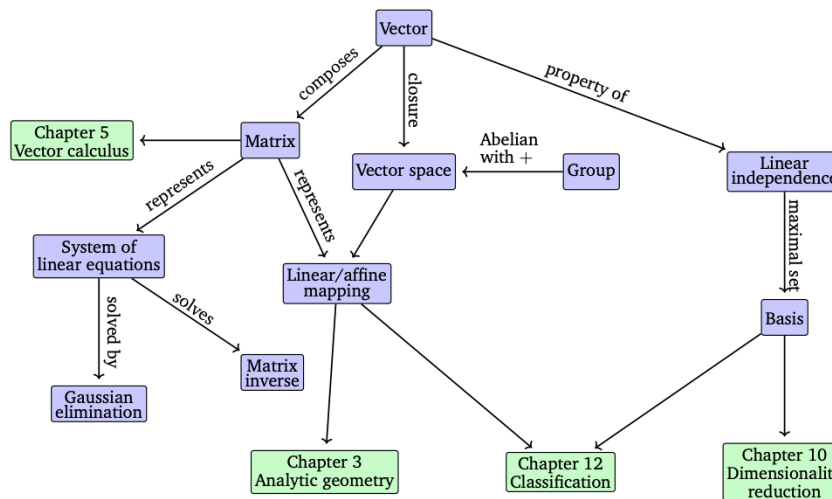
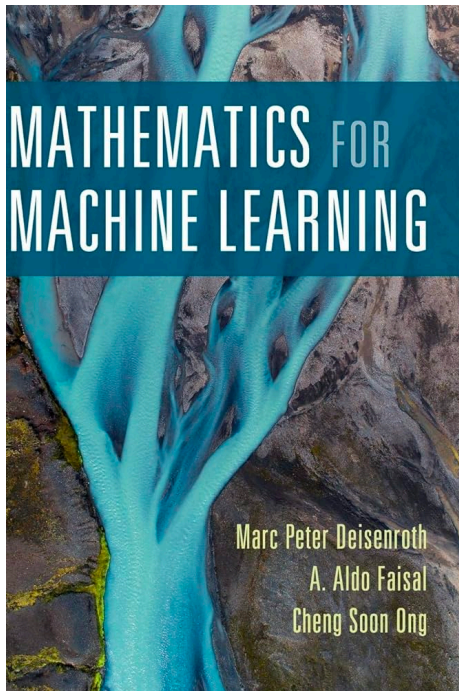
An important observation is the similarity, on a pixel level, in color and response to first and second order Gaussian filters for the red lesions and the vasculature. To obtain the feature vectors, filter outputs at each pixel location of I_{pp} were used. These filters consisted of all Gaussian derivatives up to and including second order at scales $\sigma = 1, 2, 4, 8, 16$ pixels. Also the pixel intensities taken from I_{pp} were added to the feature vector. Preliminary experiments comparing several classifiers showed that a kNN classifier [15] produced the best results. For all experiments, $k = 55$.

The kNN classifier (with $k > 1$) allows for soft-classification in which each pixel is assigned a probability of it being a foreground pixel. To accomplish this, all k neighbors in the feature space of a query pixel were examined. When n neighbors were labeled as being a foreground pixel, the posterior probability that the query pixel is a foreground pixel itself p was determined by

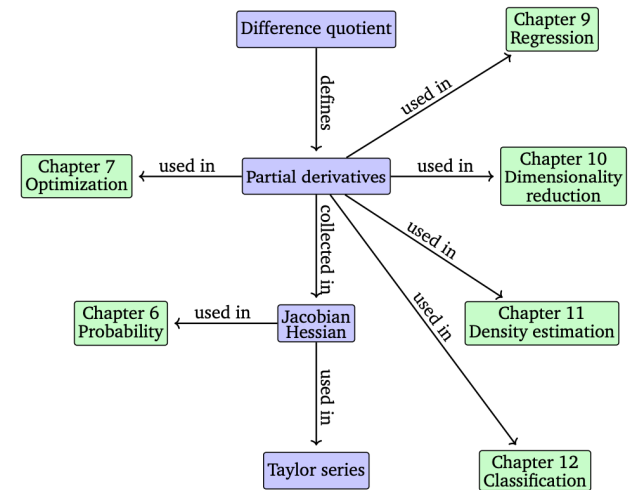
$$p = \frac{n}{k}. \quad (2)$$

Math underlying AI/ML

To understand AI/ML algorithms, we need to understand their underlying mathematical foundations.



Linear Algebra



Vector Calculus

Lecture Outline

- Mathematical foundations of AI/ML
- Back to basics:
 - **What is data? How is it generated?**
 - How do we describe multiple observations?
- What is probabilistic modeling?
- Maximum likelihood estimation
 - Closed-form solutions

What is data?

- Suppose you are given the task of predicting hospital readmission risk.
- In your dataset, you have:

- Patient age

$$X \in \mathbb{R}$$

Continuous random variable

- Patient biological sex

$$X \in \{\mathbf{male, female}\}$$

Discrete random variable

- Patient blood type

$$X \in \{A, B, AB, O\}$$

Discrete random variable

- Patient diagnoses $X = (\mathbf{has\ diabetes?, \dots, has\ heart\ failure?})$

Discrete random vector

- Whether a patient was readmitted

$$Y \in \{Yes, No\}$$

Discrete random variable (binary)

Probability distribution

DEFINITION: Probability distribution

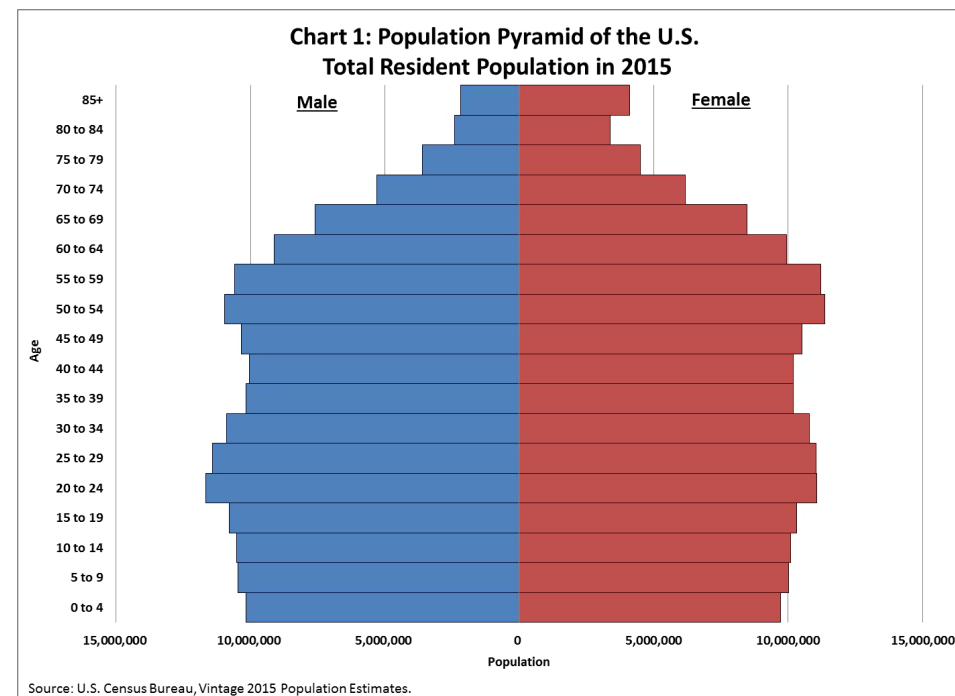
a mathematical function p that gives the probabilities of occurrence of all different possible values Ω for a random variable/vector

- Probability mass function typically refers to that for a *discrete* random variable
- Probability density function typically refers to that for a *continuous* random variable

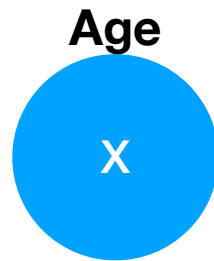
- $p(x)$ must satisfy:

- $p(x) \geq 0$

- $\sum_{x \in \Omega} p(x) = 1$ or $\int_{x \in \Omega} p(x) dx = 1$



Joint distribution

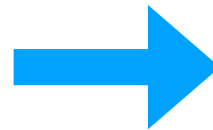


$$p(X = x, Y = y)$$

Upper case = random variable
Lower case = value

Population

X	Y
10	1
10	0
10	1
20	0
...	...



X=10	Y=0	0.1
	Y=1	0.2
X=20	Y=0	0.05
	Y=1	0.1
...	...	...

Marginal distributions

Age



$$p(X = x)$$

X=10	0.3
X=20	0.15
X=30	0.05
...	...
X=90	0.1

Readmission

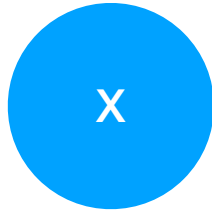


$$p(Y = y)$$

Y=1	0.2
Y=0	0.8

Conditional distributions

Age



$$p(X = x | Y = y)$$

Readmission



$$p(Y = y | X = x)$$

$Y = 0$

X=10	0.1
X=20	0.2
X=30	0.5
...	...
X=90	0.1

$Y = 1$

X=10	0.2
X=20	0.1
X=30	0.05
...	...
X=90	0.2

$X = 10$

Y=1	0.1
Y=0	0.9

$X = 90$

Y=1	0.5
Y=0	0.5

How is the joint distribution related to the marginal distribution?

How is $p(X = x, Y = y)$ related to $p(X = x)$?

The sum rule:

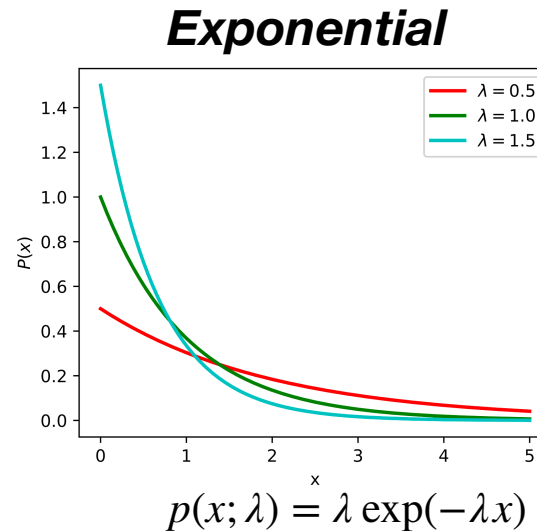
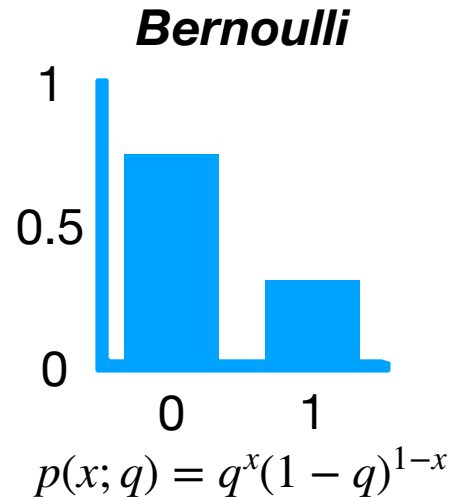
$$p(\mathbf{x}) = \begin{cases} \sum_{\mathbf{y} \in \mathcal{Y}} p(\mathbf{x}, \mathbf{y}) & \text{if } \mathbf{y} \text{ is discrete} \\ \int_{\mathcal{Y}} p(\mathbf{x}, \mathbf{y}) d\mathbf{y} & \text{if } \mathbf{y} \text{ is continuous} \end{cases}$$

The product rule:

$$p(x, y) = p(y | x)p(x)$$

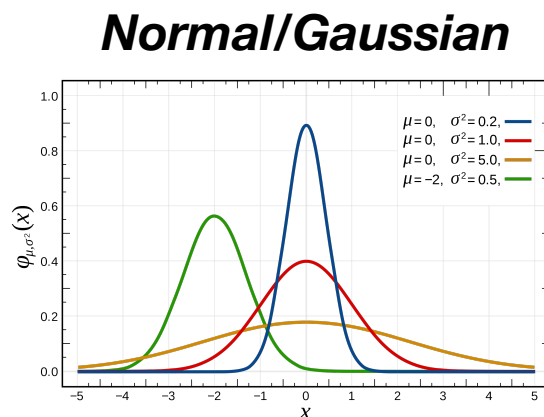
Well-known probability distributions

Name that distribution!

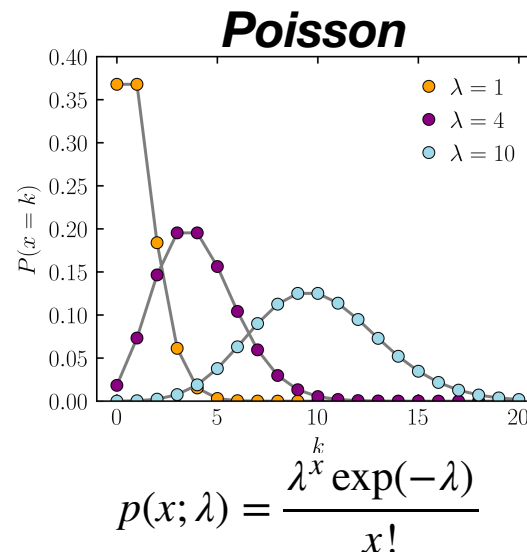


To define a specific probability distribution, we have to specify:

- the functional form
- the parameters



$$p(x; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \left(\frac{x - \mu}{\sigma}\right)^2\right)$$



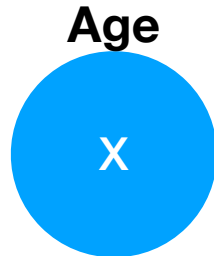
Quiz

- Q: The distribution of a binary random variable must follow that of a...
 - A: Bernoulli distribution
 - B: Multinomial distribution
 - C: Gaussian distribution
 - D: Poisson distribution
 - E: None of the above

Quiz

- Q: The distribution of a continuous random variable must follow that of a...
- A: Bernoulli distribution
- B: Multinomial distribution
- C: Gaussian distribution
- D: Poisson distribution
- E: None of the above

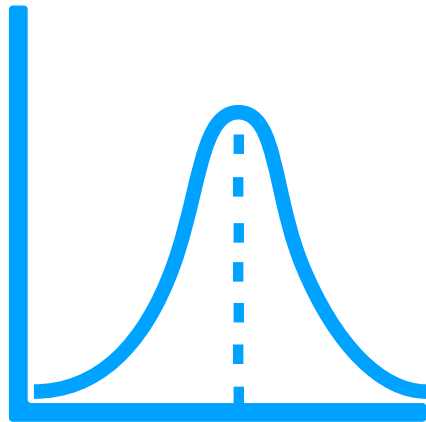
Marginal distributions



Parameters

$$p(X = x; \theta)$$

Probability
density
function



$$p(X = x) = \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2}(x - 50)^2 \right)$$

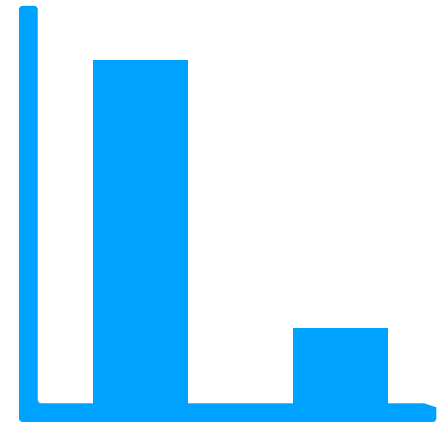
Readmission



Parameters

$$p(Y = y; \beta)$$

Probability
mass
function



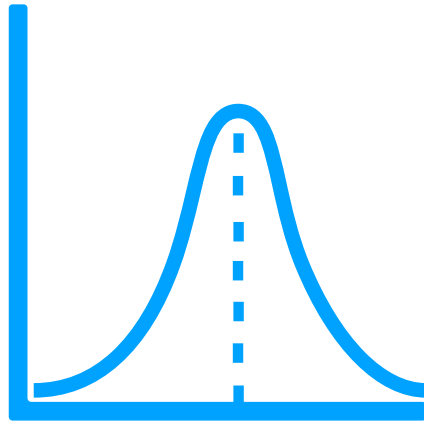
$$p(Y = y) = \begin{cases} 0.8 & \text{if } y = 1 \\ 0.2 & \text{if } y = 0 \end{cases}$$

Conditional distributions

Age



$$p(X = x | Y = y; \theta)$$

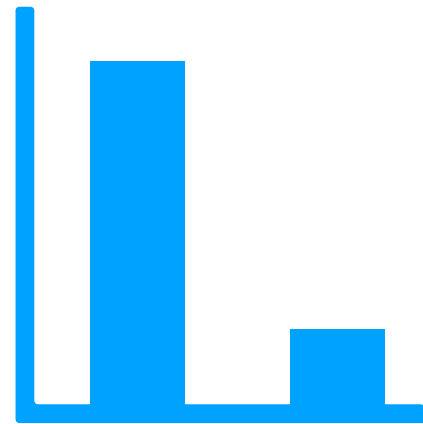


$$p(X = x | Y = y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(x - 50y)^2\right)$$

Readmission



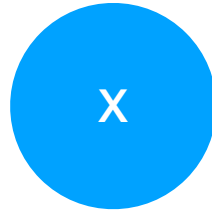
$$p(Y = y | X = x; \beta)$$



$$p(Y = 1 | X = x) = \frac{1}{1 + \exp(-0.5x)}$$

Product rule

Age



Readmission



$$p(X, Y) = p(X = x; \theta_1) \times p(Y = y | X = x; \beta_1)$$

Joint distribution *Marginal distribution* *Conditional distribution*

Discriminative models:
e.g. Random forests, Logistic regression

$$p(X, Y) = p(X = x | Y = y; \theta_2) \times p(Y = y; \beta_2)$$

Joint distribution *Conditional distribution* *Marginal distribution*

Generative models:
e.g. Linear discriminant analysis, LLMs

Quiz

- Let's consider different ways of factorizing $p(X_1, X_2, Y)$. Which of these are correct:
- A: $p(X_1)p(X_2 | X_1)p(Y | X_2)$
- B: $p(X_1)p(X_2 | X_1)p(Y | X_1, X_2)$
- C: $p(Y)p(X_2 | Y)p(X_1 | X_2, Y)$
- D: $p(Y)p(X_2 | Y)p(X_2 | X_1)$

Bayes' rule

Bayes' rule relates the two conditional distributions:

$$p(y | x) = \frac{p(x | y)p(y)}{p(x)}$$

Proof:

$$p(y | x)p(x) = p(x, y) = p(x | y)p(y) \quad \textit{Product rule}$$

Why should you care about Bayes' rule?

- Some ML algorithms take advantage of Bayes' rule, because it can sometimes be easier to learn one conditional distribution rather than another.

We can mix and match rules!

- Suppose we know $p(X_1, X_2, Y)$. How will we determine $p(X_1 | Y)$?

$$\begin{aligned} p(x_1 | y) &= \frac{p(x_1, y)}{p(y)} && \text{Bayes' rule} \\ &= \frac{\sum_{x_2} p(x_1, x_2, y)}{p(y)} && \text{Sum rule} \end{aligned}$$

Independence

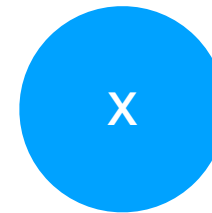
- Two variables are statistically independent if $p(x, y) = p(x)p(y)$.

- Also written as: $X \perp\!\!\!\perp Y$

- Two variables are conditionally independent given z if $p(x, y | z) = p(x | z)p(y | z)$.

- Also written as: $X \perp\!\!\!\perp Y | Z$

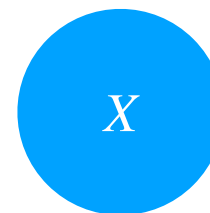
Coin Toss 1



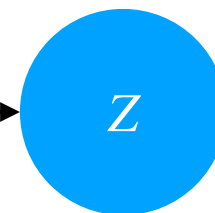
Coin Toss 2



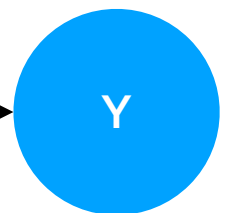
Soda Tax



Soda Buying



Blood Glucose



Lecture Outline

- Mathematical foundations of AI/ML
- Back to basics:
 - What is data? How is it generated?
 - **How do we describe multiple observations?**
- What is probabilistic modeling?
- Maximum likelihood estimation
 - Closed-form solutions

Multiple observations

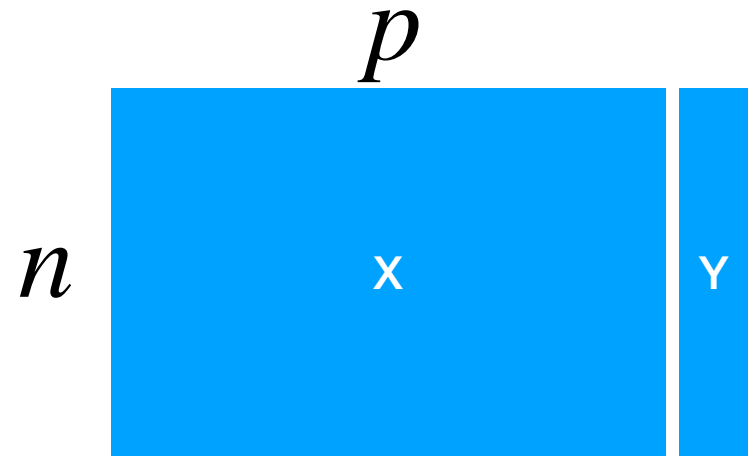
- When you have multiple observations, we need to think carefully about how the data is being generated.

- For example:

- Are observations independent and identically distributed?
- Do we have repeated observations?

- Why should you care?

Big data = multiple observations



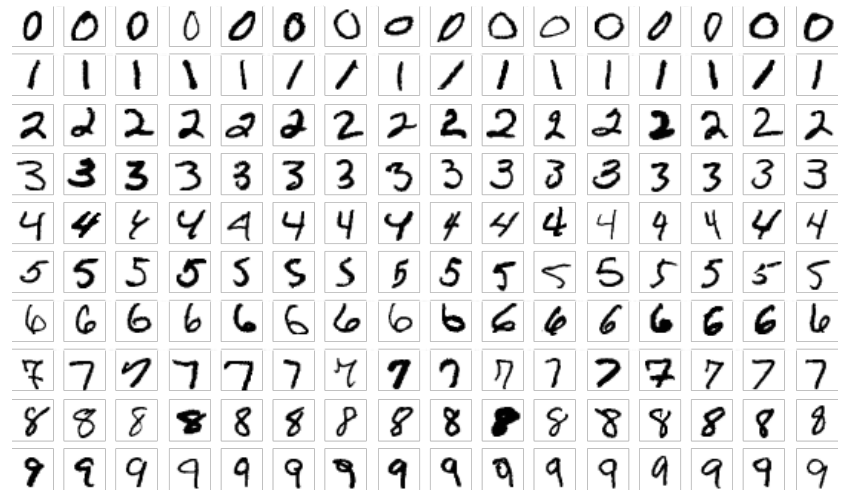
The IID assumption

- **The IID assumption:** Observations are identical and independently distributed.
- That is, the probability distribution of observations $\{X_i : i = 1, \dots, n\}$ can be written as $p(X_1, \dots, X_n) = \prod_{i=1}^n p(X_i)$.
- By far, the most common assumption we make in ML and statistics is that the data is IID.

Why does this make sense?

The IID assumption

- Examples where the IID assumption holds



- Examples where the IID assumption does not hold

Hospital readmission



Readmitted May 2019

Readmitted again June 2019

Lecture Outline

- Mathematical foundations of machine learning
- Back to basics:
 - What is data? How is it generated?
 - How do we describe multiple observations?
- **What is probabilistic modeling?**
- Maximum likelihood estimation
 - Closed-form solutions

What is probabilistic modeling?

- In machine learning, we are frequently concerned with the interpretation and analysis of data, e.g., for prediction of future events and decision making.
- We typically want the models that describe the data generating mechanism as accurately as possible.

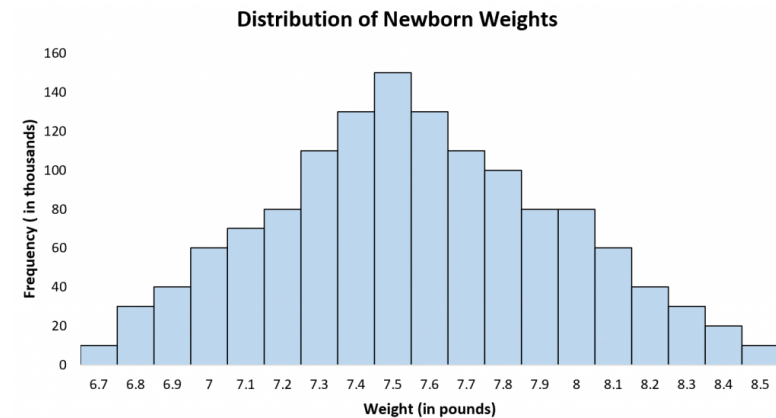
True data generating mechanism

$$p(X, Y; \theta)$$

Probability Model

$$p(X, Y; \hat{\theta})$$

- Probability models are simply functions that humans define and can compute from.
- Modeling the data helps us learn from a given dataset to *generalize to unseen data*.

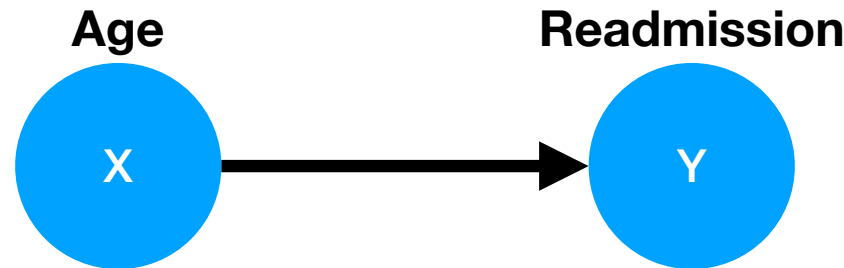


*All models are wrong
but some are useful*



George E.P. Box

How are probability models defined, mathematically?



$$p(X = x; \theta) \times p(Y = y | X = x; \beta)$$

- The probability distributions are defined according to some parameters.
 - **Parametric model:** parameters are finite dimensional.
 - **Non-parametric model:** parameters are infinite dimensional.
 - **Semi-parametric model:** a mix of finite-dimensional and infinite-dimensional parameters are used to define the probability model.

How are probability models evaluated, mathematically?

- How do we evaluate whether a population is accurately described by some model parameters θ ?
- Given a dataset from this population, we can assess how likely it would have been to observe this dataset according to the model.
- **Definition:** The likelihood of observing data $(X_1, \dots, X_n)$ according to some model parameterized by θ is
$$\mathcal{L}(\theta | X_1, \dots, X_n) = p(X_1, \dots, X_n; \theta)$$
- Assuming the data is IID, this simplifies further:
$$\mathcal{L}(\theta | X_1, \dots, X_n) = \prod_{i=1}^n p(X_i; \theta).$$

There are many flavors of probabilistic models

- We don't have to model the whole thing, e.g. Regression models only aim to model $p(Y|X)$.
- We could model all the data, e.g. Generative models like LLMs model $p(X)$.
- We could even incorporate our prior knowledge into a model using Bayesian frameworks, e.g. We have prior belief on what the most likely values are for parameter θ .
- Many statistical methods all boil down to finding a model where the probability of observing the data is maximized.

Lecture Outline

- Mathematical foundations of machine learning
- Back to basics:
 - What is data? How is it generated?
 - How do we describe multiple observations?
- What is probabilistic modeling?
- **Maximum likelihood estimation**
 - Closed-form solutions

Maximum likelihood estimation (MLE)

- MLE is one of the most fundamental concepts in statistics and one of the most commonly used probabilistic modeling techniques.
- **Definition:** The likelihood of observing data $(X_1, \dots, X_n)$ according to some model parameterized by θ is $\mathcal{L}(\theta | X_1, \dots, X_n) = p(X_1, \dots, X_n; \theta)$

- Assuming the data is IID, this simplifies further:

$$\mathcal{L}(\theta | X_1, \dots, X_n) = \prod_{i=1}^n p(X_i; \theta).$$

- Often we consider the **log likelihood** because it is easier to work with mathematically: $\log \mathcal{L}(\theta; X_1, \dots, X_n) = \sum_{i=1}^n \log p(X_i; \theta).$

- Why should you care? Because many ML algorithms use the (negative) log likelihood as a scoring/objective function. (AKA the cross-entropy loss)

- MLE tries to estimate a model by finding one that maximizes the likelihood.

$$\hat{\theta}_n = \arg \max_{\theta} \sum_{i=1}^n \log p(X_i; \theta)$$

Is maximizing the likelihood the same as maximizing the log likelihood?

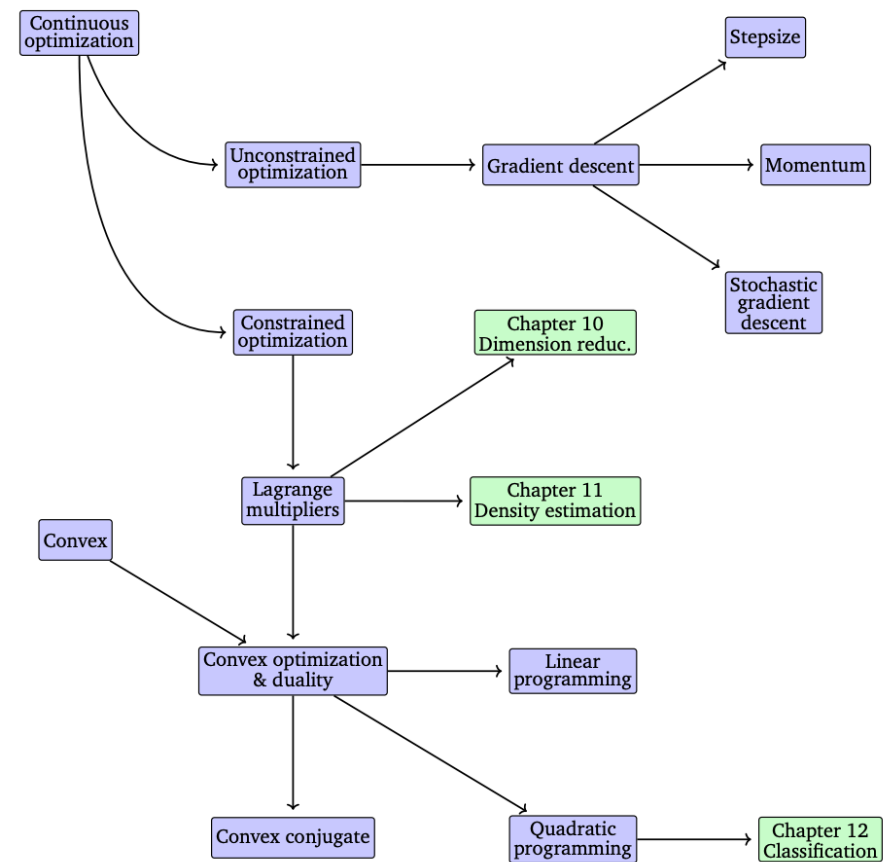
MLE

Objective function

$$\max_{\theta \in \Theta} \sum_{i=1}^n \log p(X_i; \theta)$$

This is an optimization problem.

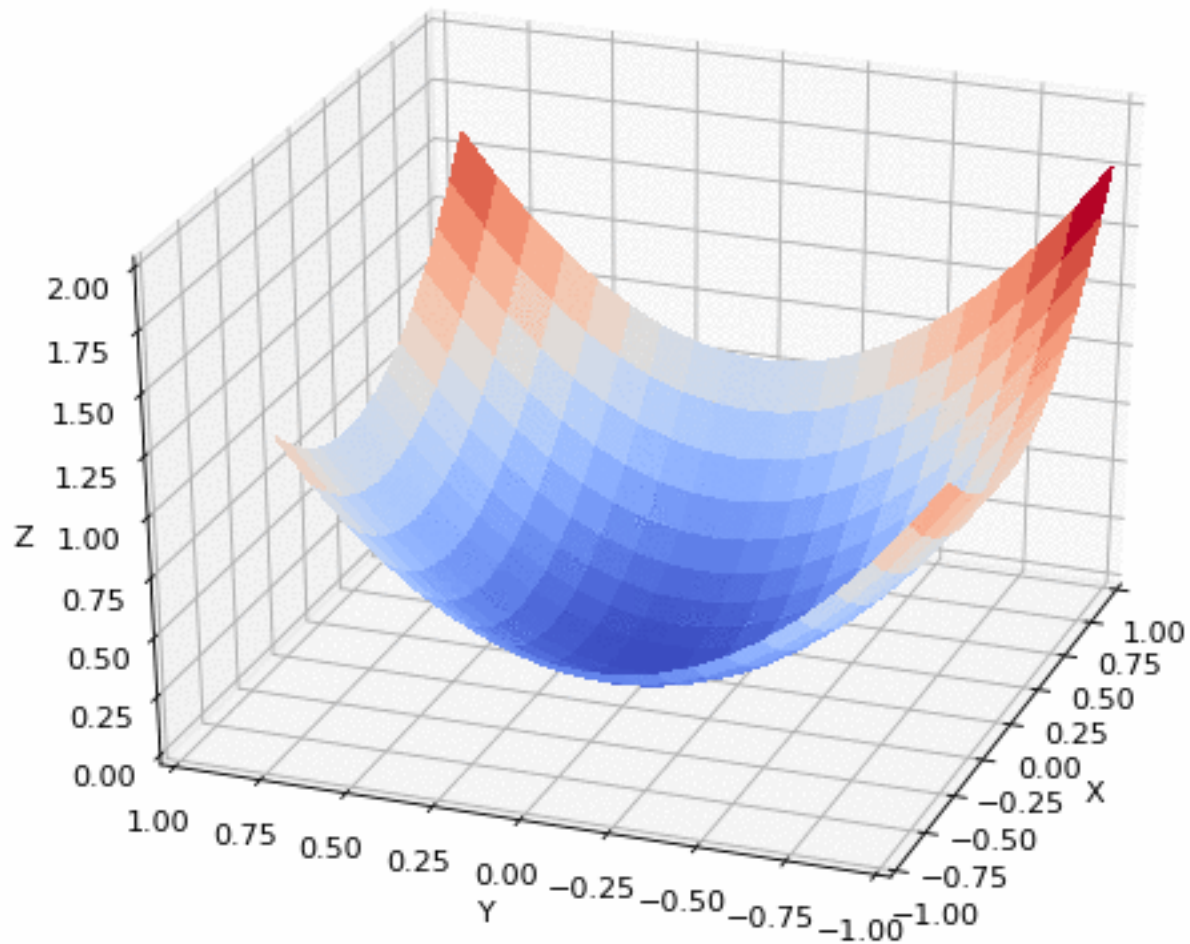
The role of optimization in ML



What methods use MLE?

- Linear Discriminant Analysis
- Linear regression
- Logistic regression
- Penalized regression
- Cox proportional hazards
- Neural networks, LLMs
- Gradient Boosted Trees

How are optimization problems solved?



How are optimization problems solved?

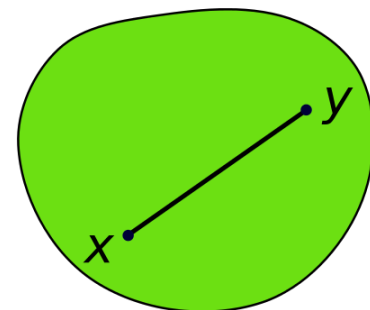
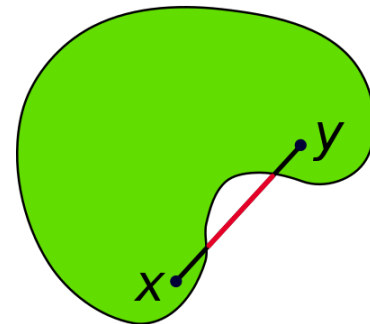
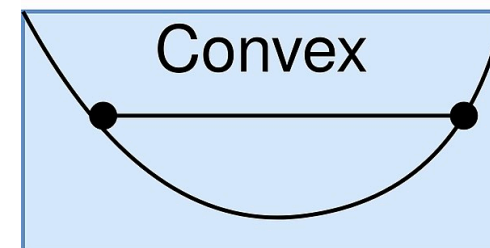
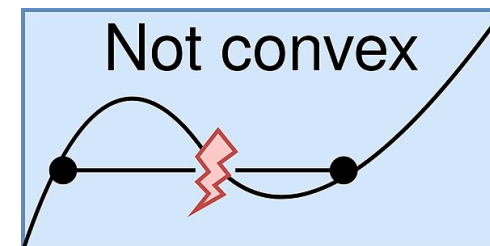
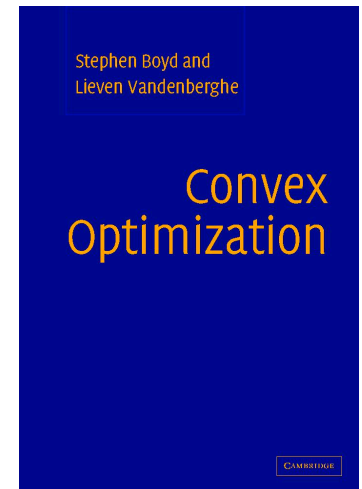
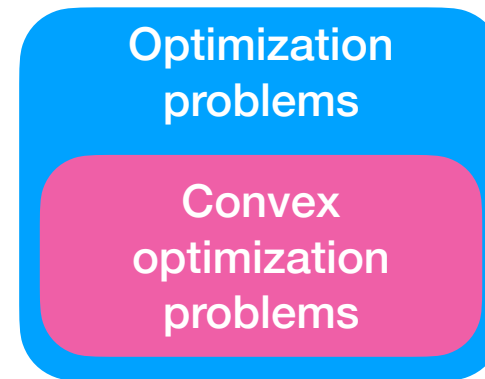
$$\hat{\theta}_n = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log p(X_i; \theta)$$

- Sometimes there is a **closed-form solution**.
- Sometimes we use an **iterative algorithm**.
- Sometimes we use **brute force**.
- Sometimes we can only provide **approximate** solutions.

When designing new machine learning algorithms, a major component is to figure out how we can solve the optimization problem as quickly and as accurately as possible.

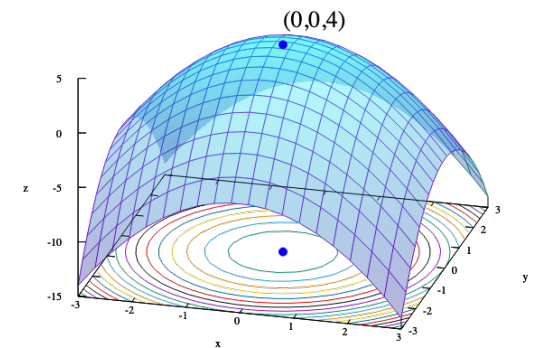
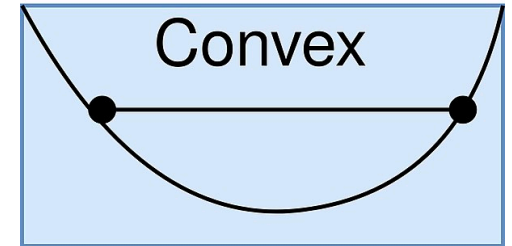
Convex optimization

- **Convex optimization:** subfield of mathematical optimization that studies the problem of minimizing **convex functions** over **convex sets**
- **Convex function:** a function f is convex if the line segment between any two distinct points on the graph of the function lies above the graph between the two points, i.e.
$$f(\alpha x_1 + (1 - \alpha)x_2) \leq \alpha f(x_1) + (1 - \alpha)f(x_2)$$
- **Convex set:** set A is convex if the whole line segment connecting any two points in A is also within A
- **Why do we study convex optimization?**
Because the optimization problem has nice properties that allow for fast and accurate solutions.



Concave optimization?

- In convex optimization, the task is to **minimize** a convex function.
- We have been talking about **maximum** likelihood.
- How are we going to use convex optimization methods to solve maximum likelihood?
 - Assuming the objective function is **concave**, you can just negate it!



Lecture Outline

- Mathematical foundations of AI/ML
- Back to basics:
 - What is data? How is it generated?
 - How do we describe multiple observations?
- What is probabilistic modeling?
- Maximum likelihood estimation
 - **Closed-form solutions**

Reminder: Gradients

- Math recap: Gradients and partial derivatives

Definition 5.5 (Partial Derivative). For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \mapsto f(\mathbf{x})$, $\mathbf{x} \in \mathbb{R}^n$ of n variables $x_1, \dots, x_n$ we define the *partial derivatives* as

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2, \dots, x_n) - f(\mathbf{x})}{h} \\ &\vdots \\ \frac{\partial f}{\partial x_n} &= \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{n-1}, x_n + h) - f(\mathbf{x})}{h} \end{aligned} \quad (5.39)$$

and collect them in the row vector

$$\nabla_{\mathbf{x}} f = \text{grad} f = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \frac{\partial f(\mathbf{x})}{\partial x_2} \quad \dots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right] \in \mathbb{R}^{1 \times n}, \quad (5.40)$$

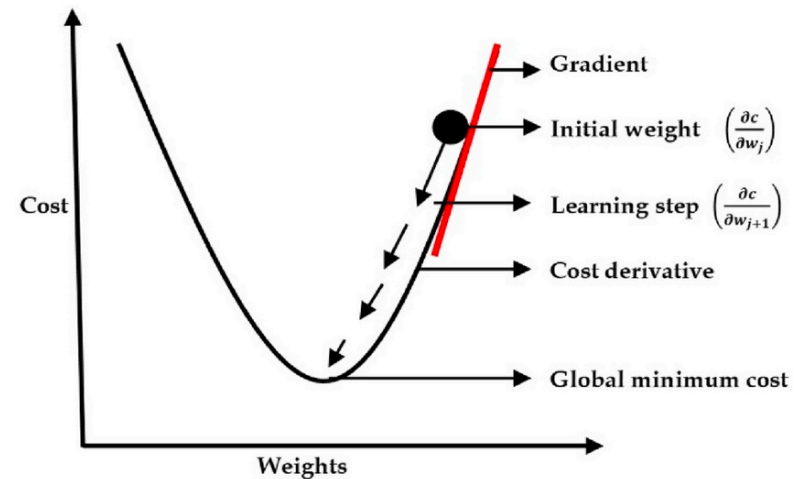
- The gradient is the direction of steepest ascent. It is also orthogonal to the level sets of the function.

First-order optimality conditions

For an unconstrained convex optimization problem

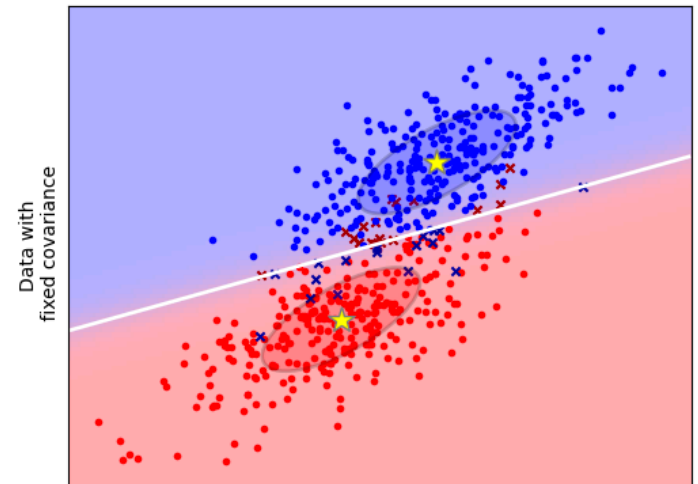
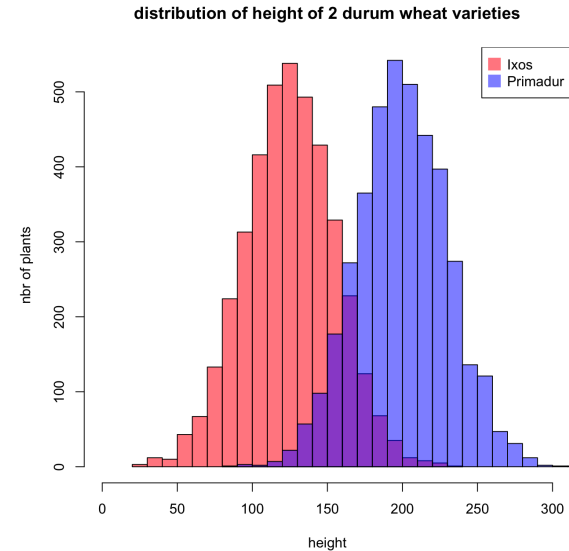
$\min_{\theta \in \mathbb{R}^p} f(\theta)$, the point θ is the solution if and only if it satisfies $\nabla_{\theta} f(\theta) = 0$.

For certain optimization problems, we can use this property to derive a closed-form solution.



Example: Linear discriminant analysis

- We have two classes $Y = 1$ and $Y = 0$ with prevalence given by $\Pr(Y = 1) = \theta$.
- We assume that variables X are normally distributed within each class, but with different means:
 - $X | Y = y \sim N(\mu_y, \sigma^2)$
- The optimization problem:
 - $$\max_{\mu_0, \mu_1, \sigma^2, \theta} \sum_{i=1}^n \log p(X = x_i, Y = y_i; \mu_0, \mu_1, \sigma^2, \theta)$$
- As we will show, this optimization problem has a closed-form solution.



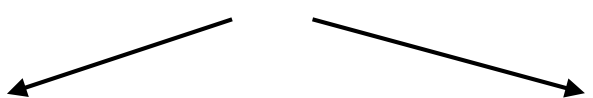
Example: Linear discriminant analysis

- To estimate μ_0, μ_1, σ^2 , and θ , we use MLE:

$$\max_{\mu_0, \mu_1, \sigma^2, \theta} \sum_{i=1}^n \log p(X = x_i, Y = y_i; \mu_0, \mu_1, \sigma^2, \theta)$$

$$= \max_{\mu_0, \mu_1, \sigma^2, \theta} \sum_{i=1}^n \log \left[p(X = x_i | Y = y_i; \mu_{y_i}, \sigma^2) p(Y = y_i; \theta) \right]$$

$$= \max_{\mu_0, \mu_1, \sigma^2, \theta} \sum_{i=1}^n \log p(X = x_i | Y = y_i; \mu_{y_i}, \sigma^2) + \log p(Y = y_i; \theta)$$


$$= \max_{\mu_0, \mu_1, \sigma^2} \sum_{i=1}^n \log p(X = x_i | Y = y_i; \mu_{y_i}, \sigma^2) + \max_{\theta} \sum_{i=1}^n \log p(Y = y_i; \theta)$$

?

?

Example: Linear discriminant analysis

First, let's solve for θ

$$\begin{aligned}\hat{\theta} &= \arg \max_{\theta} \sum_{i=1}^n \log p(Y = y_i; \theta) \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n Y_i \log \theta + (1 - Y_i) \log(1 - \theta)\end{aligned}$$

This is a convex optimization problem. The solution is where the gradient is equal to zero, i.e.

$$\begin{aligned}\left. \frac{d}{d\theta} \frac{1}{n} \sum_{i=1}^n Y_i \log \theta + (1 - Y_i) \log(1 - \theta) \right|_{\hat{\theta}} &= 0 \\ \Rightarrow \sum_{i=1}^n Y_i \frac{1}{\hat{\theta}} - (1 - Y_i) \frac{1}{1 - \hat{\theta}} &= 0 \\ \Leftrightarrow \hat{\theta} &= \frac{1}{n} \sum_{i=1}^n Y_i\end{aligned}$$

Example: Linear discriminant analysis

$$\max_{\mu_0, \mu_1, \sigma^2} \sum_{i=1}^n \log p(X = x_i | Y = y_i; \mu_0, \mu_1, \sigma^2)$$

?

$$\max_{\theta} \sum_{i=1}^n \log p(Y = y_i; \theta)$$



$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n Y_i$$

Example: Linear discriminant analysis

Next, let's solve for μ_0, μ_1, σ^2

$$\begin{aligned} & \max_{\mu_0, \mu_1, \sigma^2 > 0} \sum_{i=1}^n \log p(X = x_i | Y = y_i; \mu_{y_i}, \sigma^2) \\ \Leftrightarrow & \max_{\mu_0, \mu_1, \sigma^2 > 0} \sum_{i=1}^n (1 - y_i) \log \left\{ \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{x_i - \mu_0}{\sigma} \right)^2 \right) \right\} + y_i \log \left\{ \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{x_i - \mu_1}{\sigma} \right)^2 \right) \right\} \\ \Leftrightarrow & \max_{\mu_0, \mu_1, \sigma^2 > 0} \sum_{i=1}^n -\log \sigma - \frac{1}{2} (1 - y_i) \left(\frac{x_i - \mu_0}{\sigma} \right)^2 - \frac{1}{2} y_i \left(\frac{x_i - \mu_1}{\sigma} \right)^2 \\ \Leftrightarrow & \min_{\mu_0, \mu_1, \sigma^2 > 0} \sum_{i=1}^n \log \sigma + \frac{1}{2} (1 - y_i) \left(\frac{x_i - \mu_0}{\sigma} \right)^2 + \frac{1}{2} y_i \left(\frac{x_i - \mu_1}{\sigma} \right)^2 \end{aligned}$$

Example: Linear discriminant analysis

Next, let's solve for μ_0 by finding the value for which the gradient is equal to zero:

$$\left. \frac{d}{d\mu_0} \sum_{i=1}^n \log \sigma + (1 - y_i) \left(\frac{x_i - \mu_0}{\sigma} \right)^2 + y_i \left(\frac{x_i - \mu_1}{\sigma} \right)^2 \right|_{\hat{\mu}_0} = 0$$

$$\left. \frac{d}{d\mu_0} \sum_{i=1}^n (1 - y_i) \left(\frac{x_i - \mu_0}{\sigma} \right)^2 \right|_{\hat{\mu}_0} = 0$$

$$\sum_{i=1}^n (1 - y_i)(x_i - \hat{\mu}_0) = 0$$

$$\hat{\mu}_0 = \frac{\sum_{i=1}^n x_i(1 - y_i)}{\sum_{i=1}^n (1 - y_i)}$$

Example: Linear discriminant analysis

Next, let's solve for σ^2

$$\left. \frac{d}{d\sigma^2} \sum_{i=1}^n \frac{1}{2} \log \sigma^2 + \frac{1}{2}(1 - y_i) \left(\frac{x_i - \mu_0}{\sigma} \right)^2 + \frac{1}{2} y_i \left(\frac{x_i - \mu_1}{\sigma} \right)^2 \right|_{\hat{\sigma}^2} = 0$$

$$\sum_{i=1}^n \frac{1}{2\hat{\sigma}^2} - (1 - y_i)(x_i - \mu_0)^2 \frac{1}{2\hat{\sigma}^4} - y_i(x_i - \mu_1)^2 \frac{1}{2\hat{\sigma}^4} = 0$$

$$\sum_{i=1}^n \hat{\sigma}^2 - (1 - y_i)(x_i - \mu_0)^2 - y_i(x_i - \mu_1)^2 = 0$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_0(1 - y_i) - \mu_1 y_i)^2$$

Example: Linear discriminant analysis

$$\max_{\mu_0, \mu_1, \sigma^2} \sum_{i=1}^n \log p(X = x_i | Y = y_i; \mu_0, \mu_1, \sigma^2)$$



$$\hat{\mu}_0 = \frac{\sum_{i=1}^n x_i(1 - y_i)}{\sum_{i=1}^n (1 - y_i)}$$

$$\hat{\mu}_1 = \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n y_i}$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_0(1 - y_i) - \mu_1 y_i)^2$$

$$\max_{\theta} \sum_{i=1}^n \log p(Y = y_i; \theta)$$



$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^n Y_i$$

Example: Linear discriminant analysis

- What you see in scikit-learn:

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

```
class sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver='svd', shrinkage=None, priors=None,
n_components=None, store_covariance=False, tol=0.0001, covariance_estimator=None) \[source\]
```

Linear Discriminant Analysis.

A classifier with a linear decision boundary, generated by fitting class conditional densities to the data and using Bayes' rule.

The model fits a Gaussian density to each class, assuming that all classes share the same covariance matrix.

The fitted model can also be used to reduce the dimensionality of the input by projecting it to the most discriminative directions, using the `transform` method.

New in version 0.17: LinearDiscriminantAnalysis.

Read more in the [User Guide](#).

```
>>> import numpy as np
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
>>> y = np.array([1, 1, 1, 2, 2, 2])
>>> clf = LinearDiscriminantAnalysis()
>>> clf.fit(X, y)
LinearDiscriminantAnalysis()
>>> print(clf.predict([[-0.8, -1]]))
[1]
```

Example: Linear discriminant analysis

- What is happening behind in the scenes in scikit-learn:

https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/discriminant_analysis.py

```
def _class_means(X, y):
    """Compute class means.

    Parameters
    -----
    X : array-like of shape (n_samples, n_features)
        Input data.

    y : array-like of shape (n_samples,) or (n_samples, n_targets)
        Target values.

    Returns
    -----
    means : array-like of shape (n_classes, n_features)
        Class means.
    """
    xp, is_array_api_compliant = get_namespace(X)
    classes, y = xp.unique_inverse(y)
    means = xp.zeros((classes.shape[0], X.shape[1]), device=device(X), dtype=X.dtype)

    if is_array_api_compliant:
        for i in range(classes.shape[0]):
            means[i, :] = xp.mean(X[y == i], axis=0)
    else:
        # TODO: Explore the choice of using bincount + add.at as it seems sub optimal
        # from a performance-wise
        cnt = np.bincount(y)
        np.add.at(means, y, X)
        means /= cnt[:, None]
    return means
```

```
def _class_cov(X, y, priors, shrinkage=None, covariance_estimator=None):
    """Compute weighted within-class covariance matrix.

    The per-class covariance are weighted by the class priors.

    Parameters
    -----
    X : array-like of shape (n_samples, n_features)
        Input data.

    y : array-like of shape (n_samples,) or (n_samples, n_targets)
        Target values.

    priors : array-like of shape (n_classes,)
        Class priors.

    shrinkage : 'auto' or float, default=None
        Shrinkage parameter, possible values:
        - None: no shrinkage (default).
        - 'auto': automatic shrinkage using the Ledoit-Wolf lemma.
        - float between 0 and 1: fixed shrinkage parameter.

    Shrinkage parameter is ignored if 'covariance_estimator' is not None.

    covariance_estimator : estimator, default=None
        If not None, 'covariance_estimator' is used to estimate
        the covariance matrices instead of relying the empirical
        covariance estimator (with potential shrinkage).
        The object should have a fit method and a ``covariance_`` attribute
        like the estimators in sklearn.covariance.
        If None, the shrinkage parameter drives the estimate.

    .. versionadded:: 0.24

    Returns
    -----
    cov : array-like of shape (n_features, n_features)
        Weighted within-class covariance matrix
    """
    classes = np.unique(y)
    cov = np.zeros(shape=(X.shape[1], X.shape[1]))
    for idx, group in enumerate(classes):
        Xg = X[y == group, :]
        cov += priors[idx] * np.atleast_2d(_cov(Xg, shrinkage, covariance_estimator))
    return cov
```

```
def _solve_lstsq(self, X, y, shrinkage, covariance_estimator):
    """Least squares solver.
```

The least squares solver computes a straightforward solution of the optimal decision rule based directly on the discriminant functions. It can only be used for classification (with any covariance estimator), because estimation of eigenvectors is not performed. Therefore, dimensionality reduction with the transform is not supported.

Parameters

X : array-like of shape (n_samples, n_features)
Training data.

y : array-like of shape (n_samples,) or (n_samples, n_classes)
Target values.

shrinkage : 'auto', float or None
Shrinkage parameter, possible values:
- None: no shrinkage.
- 'auto': automatic shrinkage using the Ledoit-Wolf lemma.
- float between 0 and 1: fixed shrinkage parameter.

Shrinkage parameter is ignored if 'covariance_estimator' is not None

covariance_estimator : estimator, default=None

If not None, 'covariance_estimator' is used to estimate the covariance matrices instead of relying the empirical covariance estimator (with potential shrinkage). The object should have a fit method and a ``covariance\_`` attribute like the estimators in sklearn.covariance. If None the shrinkage parameter drives the estimate.

.. versionadded:: 0.24

Notes

This solver is based on [1]_, section 2.6.2, pp. 39–41.

References

.. [1] R. O. Duda, P. E. Hart, D. G. Stork. Pattern Classification (Second Edition). John Wiley & Sons, Inc., New York, 2001. ISBN 0-471-05669-3.

"""

```
self.means_ = _class_means(X, y)
self.covariance_ = _class_cov(
    X, y, self.priors_, shrinkage, covariance_estimator
)
self.coef_ = linalg.lstsq(self.covariance_, self.means_.T)[0].T
self.intercept_ = -0.5 * np.diag(np.dot(self.means_, self.coef_.T)) + np.log(
    self.priors_
)
```

Lecture Outline

- Mathematical foundations of AI/ML
- Back to basics:
 - What is data? How is it generated?
 - How do we describe multiple observations?
- What is probabilistic modeling?
- Maximum likelihood estimation
 - Closed-form solutions